PRODUCT REQUIREMENTS DOCUMENT

Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management

Real-Time Actual Progress Tracking — Bridging Planning and Execution across Multi-Discipline Field Operations

Document Version	2.2.0 — Enterprise High-Scale Architecture
Status	DRAFT FOR REVIEW
Primary Stack	React.js · FastAPI · PostgreSQL + pgvector · Redis + Celery
Target Domain	Infrastructure / EPC Project Management (Civil, Piping, Static & Rotating Equipment, Electrical, Instrumentation, HSE)
Prepared By	Shreyas S
Date	August 2026

< 15 min reconciliation lag (from 7–14 days)

≥ 92% automated matching precision

≥ 80% reduction in planner data-entry overhead

100% audit traceability, zero silently dropped events

Zero-trust security · OWASP Top 10 hardened

WHAT'S INSIDE

Architecture & Data Flow
Multi-tier system design, ingestion pipeline, and confidence-based routing (§7–9)

Security & Scale
Zero-trust architecture, RBAC, and high-throughput optimization (§10–12)

Delivery & Roadmap
Testing, deployment, risk register, and the CCTV ingestion roadmap (§14–19)

Table of Contents

Document Control

Document Title	Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management
Version	2.2.0-PROD (Enterprise High-Scale Architecture)
Tech Stack	React.js (Frontend) · FastAPI (Backend) · PostgreSQL 16 + pgvector (Database) · Redis + Celery (Async Processing)
Target Profile	Cloud-Native · Zero-Trust Security · High-Throughput Ingestion

Revision History

Version	Date	Change Summary
2.0.0	Aug 2026	Initial enterprise architecture: multi-modal ingestion, neuro-symbolic matching, confidence router, security & scale design.
2.1.0	Aug 2026	Re-scoped against formal problem statement. Added Conversational Time-Agent, granularity-aware progress rollup, new-activity discovery path, multi-contractor/ discipline modeling, institutional knowledge schema, NFRs, observability, testing, deployment, risk register, traceability matrix, and CCTV ingestion roadmap.
2.2.0	Aug 2026	Reconciled API design (§8) against the working prototype's endpoint list: added health/readiness, schedule bootstrap-upload, activity listing, batch spreadsheet ingestion, event listing, and agent session-history endpoints. Flagged that the planner match-resolve endpoint is prototype-critical and must not be dropped.

> **What changed in this revision:** Nothing from the original v2.0 scope was removed. This revision closes gaps between the original technical spec and the problem statement it must solve — it is strictly additive.

1. Executive Summary

Infrastructure and EPC (Engineering, Procurement, Construction) schedules cascade from macro milestones (L1) down to thousands of granular, executable L5/L6 activities spanning parallel disciplines — civil, piping, static and rotating equipment, electrical, instrumentation, and HSE. While the baseline plan lives cleanly in Primavera P6 or MS Project, **actual execution data does not flow back cleanly**: it arrives as free-text daily reports, discipline-wise spreadsheets, scanned site diaries, and verbal supervisor updates — each in its own format, cadence, and vocabulary, and largely disconnected from L5/L6 activity IDs.

This system is the **planning-to-execution bridge**: it ingests heterogeneous field inputs, extracts structured activity-level events using LLM function calling, fuzzy-matches them to the correct WBS node using a hybrid neuro-symbolic engine (semantic vector similarity + deterministic schedule-rule validation), and auto-commits high-confidence matches back to the schedule/PMIS in near real time — with a full audit trail. Low-confidence or unmatched activities are routed to a human review queue rather than silently dropped, and every human

correction feeds an active-learning glossary that improves future matching. Over time, the system also builds a structured, cross-project **institutional knowledge base** of real durations, delay causes, and discipline-wise productivity — memory that today lives only in individual supervisors' heads and is lost when a project closes.

2. Problem Statement & Background

2.1 Background

Baseline schedules are well-structured artifacts. Actual progress reporting is not. Field execution is frequently **more granular** than the planned WBS (e.g., a single L5 activity like "Erect Line 24"-XX" may be reported by the crew as several discrete sub-steps), and different disciplines describe the same physical progress differently (e.g., "spool erected" vs. the plan's official activity description). Input quality also varies with the reporting supervisor's skill, discipline convention, and format.

2.2 Problem Description

Symptom	Consequence
Actual progress data is fragmented, delayed, and inconsistently structured across disciplines and contractors	No single, trustworthy source of "what actually happened on site today"
Manual reconciliation with the baseline schedule is slow and error-prone	Schedule updates lag execution by days to weeks
Downstream performance analytics and forecasting inherit poor-quality, late data	The AI performance-monitoring stack that depends on this data is undermined at the source
Real durations, bottlenecks, and deviations are never captured in structured form	Institutional knowledge is lost at project close instead of feeding future planning

2.3 Expected Outcome

- Ingest heterogeneous discipline-wise inputs — free text, spreadsheets, scanned diaries, Primavera/MS Project exports — and extract activity-level actual start/end events.
- Offer an LLM-based conversational or voice **"time agent"** for site supervisors to log activity start/end with minimal friction, replacing rigid manual forms while still producing structured output.
- Fuzzy-match discipline-specific descriptions to the correct L5/L6 node, handling terminology and granularity mismatches, and **flag unmatched/new activities for planner review rather than silently dropping them**.
- Auto-update actual dates in the schedule/PMIS in near real time, with a confidence score and audit trail per entry.
- Produce a clean, discipline-tagged actual-progress dataset that feeds (a) live performance analytics and forecasting, and (b) a growing, queryable institutional-memory repository of real execution patterns.

3. Goals & Success Metrics

Goal	Metric	Target
Reduce schedule-update lag	Time from field event to committed WBS update (auto-committed path)	< 15 minutes, down from a 7–14 day manual baseline
Reduce planner manual reconciliation effort	% of field events auto-committed without human touch	≥ 80% reduction in manual planner data-entry overhead
Match field updates correctly	Automated matching precision, field update → correct WBS node	≥ 92%
Keep human review load manageable	Median time-to-clear the Planner Exception Dashboard queue	< 24 hours
Never silently lose data	% of low-confidence / unmatched events that are dropped rather than queued	0%
Improve matching over time	Auto-commit rate trend per project, week over week	Monotonically improving for first 8 weeks
Preserve institutional knowledge	% of closed projects with a structured durations/delay-causes dataset exported to the knowledge base	100%
Full traceability	Actual start/end updates with an immutable audit log entry	100%
System responsiveness under load	Ingest endpoint p99 latency	< 50ms (excludes AI processing, which is async)

4. Scope

4.1 In Scope

- Multi-modal field-update ingestion (voice, text, spreadsheet, scanned PDF, PMIS export).
- Conversational/voice time-agent interface for supervisors across all disciplines.
- Neuro-symbolic matching to L5/L6 WBS nodes with confidence-based auto-commit vs. human review routing.
- Granularity-aware partial-progress aggregation (many field events → one WBS node).
- New-activity discovery workflow for field-reported work with no corresponding plan node.
- Bi-directional sync with Primavera P6 / MS Project.
- Active-learning glossary per project; cross-project institutional knowledge base.
- Role-based Planner Exception Dashboard and analytics/forecasting views.
- Zero-trust security architecture, audit logging, multi-contractor access control.

4.2 Out of Scope (this phase)

- Computer-vision / CCTV-based progress detection — **designed for, not built** in this phase (see §20 Roadmap).

- Cost/commercial (EVM budget) tracking — schedule progress only, not earned value in dollars.
- Automated schedule re-baselining or critical-path re-optimization — the system reports actuals; re-planning remains a human/Primavera function.
- Native offline-first mobile data capture (site connectivity assumed intermittent-but-present; true offline queueing is a candidate for a later phase).

5. User Personas & Roles

Persona	Role Tag	Primary Needs
Site Supervisor	SUPERVISOR	Log activity start/end in seconds, in their own words, in their own discipline's vocabulary and language, via voice or chat, on a mobile device with unreliable connectivity.
Discipline Planner	PLANNER	Review low-confidence matches and unmatched/new activities; correct or approve; see per-discipline progress rollups.
Project Controls / PMO	ADMIN	Cross-discipline dashboards, forecasting, delay/risk analytics, PMIS sync health.
Contractor Admin	CONTRACTOR_ADMIN	Manage their own crews' access; see only their organization's/ discipline's data unless explicitly shared.
System Integrator / DevOps	SYSTEM	Service-to-service auth for PMIS webhooks and CI/CD pipelines.

6. Core Feature Set

6.1 Heterogeneous Multi-Modal Ingestion Engine

- Parses voice notes (Whisper ASR), scanned PDFs (OCR), free-text daily reports, Excel sheets, and Primavera/MS Project exports.
- Uses LLM function calling to extract a standardized JSON schema: `[Discipline, Object, Action, Start/End, Quantity, Contractor, Location]`.
- NEW Ingestion sources are modeled as a pluggable adapter interface rather than a fixed set — see §20 for how this lets a future CCTV/vision adapter plug into the identical downstream pipeline with zero re-architecture.

Normalized Extraction Output

Regardless of input modality, every parser converges on the same standardized JSON schema before it reaches the matching engine:

```
{
  "event_id": "evt_9842fbc8-8472",
  "project_id": "PRJ-LNG-2026-01",
  "discipline": "PIPING",
  "org_id": "org_contractor_alpha",
  "timestamp_logged": "2026-08-30T17:30:00Z",
  "source_type": "TEXT_DIARY",
  "raw_input_text": "Completed hydrostatic pressure test for line 24-B in Zone 3 today.",
```

```
  "extracted_entities": {
    "action": "HYDROSTATIC_TEST",
    "target_object": "Line 24-B",
    "location_zone": "Zone 3",
    "status": "COMPLETED",
    "actual_start": "2026-08-30T08:00:00Z",
    "actual_end": "2026-08-30T16:30:00Z",
    "quantities": [{"unit": "meters", "value": 150}]
  }
}
```

6.2 Conversational Time-Agent Interface NEW

A distinct capability from passive voice-note parsing: an LLM-driven, turn-based conversational agent (voice or chat) that supervisors interact with directly to log progress.

- Asks targeted clarifying questions when extraction confidence is low at the point of capture ("Which line number — 24 or 24A?") instead of pushing ambiguity downstream into the review queue.
- Supports multiple languages/dialects common on multi-contractor sites, with the same standardized JSON output regardless of input language.
- Designed for near-zero-friction logging: a 10–15 second voice exchange replaces a rigid form.
- Every agent session is persisted as a raw transcript in `field_events.raw_input_text` for audit and future model fine-tuning.

6.3 Hybrid Neuro-Symbolic Matching Engine

- **Semantic Pass (pgvector):** converts field descriptions into vector embeddings and calculates cosine similarity against the master WBS schedule — this is what absorbs terminology differences like "spool erected" vs. "Erect Line 24"-XX".
- **Symbolic Pass (Constraint Solver):** validates the vector match against deterministic schedule rules (predecessors complete, discipline matches, activity not already closed).
- NEW **Granularity Reconciliation:** when field reporting is finer than the planned WBS node, multiple matched events roll up against the same node with a `progress_contribution_pct`, rather than forcing a 1:1 event-to-activity assumption.

Weighted Scoring Formula

The two passes are combined into a single final confidence score used for routing:

$$S_{final} = \alpha \cdot S_{dense} + \beta \cdot S_{symbolic} \quad (\alpha = 0.70, \beta = 0.30, \text{both project-tunable})$$

S_{dense} is the pgvector cosine similarity between the field-event embedding and the WBS node embedding: $\cos(A,B) = (A \cdot B) / (\|A\| \|B\|)$. $S_{symbolic}$ is the fraction of hard constraints satisfied (discipline match, predecessor completion, actual_end ≥ actual_start).

6.4 Event-Driven Confidence Router

Confidence (S_{final})	Routing Path	Action Taken
≥ 0.85	Auto-Commit Pipeline	Writes actual start/end + progress % directly to the schedule DB and pushes to Primavera P6 / MS Project via REST/XML API; writes audit log entry.

0.60 – 0.85	Planner Review Queue (`PENDING_REVIEW`)	Displays top-3 candidate WBS nodes in the Planner UI for one-click approval or correction.
< 0.60	NEW New-Activity Queue (`NEW_ACTIVITY_CANDIDATE`)	Flagged as "Unrecognized / Out-of-Scope Work" — never dropped. Planner either links it to an existing node the matcher missed, or promotes it into a new WBS entry.

Both thresholds (0.85 and 0.60) are configurable per project and discipline via `projects.auto_commit_threshold` and `projects.min_candidate_threshold` — never globally hardcoded, since discipline-specific vocabularies converge at different rates.

6.5 Active Learning & Glossary Fine-Tuning

- Captures human corrections from the Planner Dashboard into a structured `glossary_mappings` table, mapping field jargon → official WBS activity codes, scoped per project and discipline.
- Glossary entries are promoted to prompt context / embedding fine-tuning inputs for future extraction on the same project, and contribute anonymized patterns to the cross-project glossary.

6.6 Institutional Knowledge Graph expanded

- Preserves historical execution data: actual vs. planned durations, delay reasons, discipline-wise productivity rates.
- NEW Explicitly modeled as a **cross-project, queryable repository** (not per-project silos) so a new project's planners can query "typical actual duration for piping spool erection, similar site conditions" before the project even starts.
- Structured `delay_reasons` and `productivity_benchmarks` tables populated continuously during execution and finalized at project closeout — see §10.6.

6.7 Multi-Contractor & Discipline Access Control NEW

- Field events and RBAC are scoped by `organization_id` (contractor) and discipline, so a contractor's supervisors see and log only their own scope while planners/PMO see the aggregate.

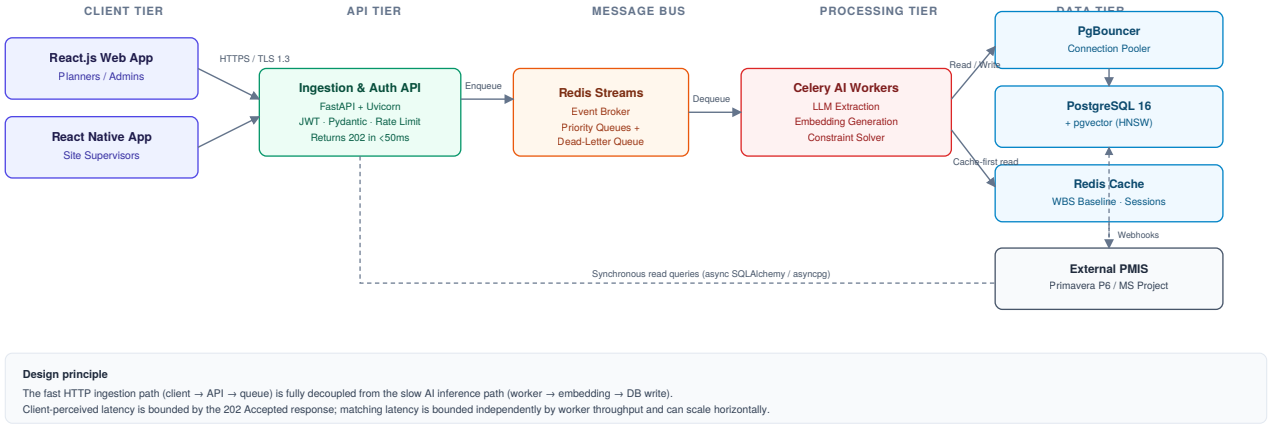

Figure 1 — High-level system architecture across client, API, messaging, processing, and data tiers.

7. System Architecture & Data Flow

To support heavy traffic and high data ingestion, the architecture fully decouples the fast HTTP ingestion layer from the slow AI inference and matching layers using asynchronous message brokers. This means client-perceived latency is bounded by a 202 Accepted response, while matching throughput scales independently by adding Celery workers.

7.1 High-Throughput Ingestion & Confidence-Routing Flow

1. **Ingest:** Client (or the Time-Agent, on the supervisor's behalf) sends a field update payload to `POST /api/v1/events/ingest`.
2. **Acknowledge:** FastAPI validates the JWT and schema via Pydantic, generates an idempotent `event_id`, pushes the payload to Redis, and returns `202 Accepted` in < 50ms.
3. **Process:** A Celery worker picks up the event, runs LLM extraction, generates the embedding, and queries pgvector for candidate WBS matches.
4. **Reconcile:** The symbolic constraint solver validates the candidate; granularity reconciliation aggregates partial progress where applicable.
5. **Route:** ≥ 0.85 → auto-commit + PMIS push. < 0.85 with a candidate node → Planner queue. No credible candidate → New-Activity queue.
6. **Record:** Every outcome — auto-committed, human-corrected, or rejected — writes an immutable audit log entry.

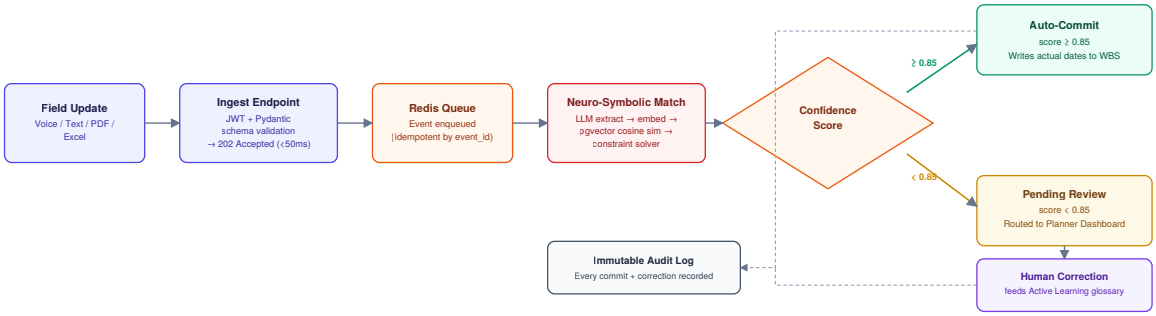

Figure 2 — Field event lifecycle from capture through confidence-based routing to audit.

8. API Design

8.1 Operational Endpoints

Method	Path	Role	Purpose
GET	`/`	—	**NEW** Basic service response — deployment sanity check
GET	`/health`	—	**NEW** Health check — backs the Kubernetes liveness/readiness probes referenced in §11.2

8.2 Schedule & Progress Endpoints

Method	Path	Role	Purpose
POST	`/api/v1/schedule/upload`	PLANNER+	**NEW** Bootstraps or re-baselines the WBS from a Primavera/MS Project export — the seed path used before, or alongside, live PMIS API sync
GET	`/api/v1/schedule/activities`	PLANNER+	**NEW** Paginated, filterable list of WBS nodes (by discipline, status) — browsing, not just single-node lookup
GET	`/api/v1/wbs/{wbs_id}/progress`	PLANNER+	Rolled-up actual progress for one WBS node, including contributing events
POST	`/api/v1/progress/extract`	SUPERVISOR+	**NEW** Synchronous dry-run: runs LLM extraction on free text and returns the structured draft *without* committing — lets the Time-Agent/UI show a preview before submission
POST	`/api/v1/events/ingest`	SUPERVISOR+	Accepts one field update (text/voice/file); returns 202 with `event_id`
POST	`/api/v1/progress/upload-excel`	SUPERVISOR+	**NEW** Batch ingestion: one discipline-wise spreadsheet fans out into many `field_events` rows, each tracked with its own `event_id`

GET	`/api/v1/progress/events`	SUPERVISOR+	**NEW** Paginated list of field events across all ingestion sources, scoped to the caller's org/discipline
GET	`/api/v1/schedule/queue`	PLANNER+	Paginated Planner Exception Dashboard queue (`PENDING_REVIEW` + `NEW_ACTIVITY_CANDIDATE`)
POST	`/api/v1/schedule/matches/{match_id}/resolve`	PLANNER+	**Critical path:** approves, corrects, or rejects a match; writes a glossary entry on correction. This is the endpoint the entire confidence-router workflow (§6.4) depends on — without it the review queue has no way to close the loop.

8.3 Conversational Agent Endpoints

Method	Path	Role	Purpose
POST	`/api/v1/agent/chat`	SUPERVISOR+	Opens or continues a Time-Agent turn (voice or chat); returns the agent's reply plus a structured draft event
GET	`/api/v1/agent/sessions/{session_id}/events`	SUPERVISOR+	**NEW** Returns every event a given agent session produced — needed to review or audit what a conversational session actually logged

8.4 Analytics, Knowledge Base & Identity

Method	Path	Role	Purpose
POST	`/api/v1/auth/login`	—	Issues short-lived JWT + refresh token (HttpOnly cookie)
POST	`/api/v1/auth/refresh`	—	Rotates refresh token, issues new JWT
GET	`/api/v1/analytics/discipline-summary`	ADMIN	Discipline-wise actual vs. planned, delay causes, productivity trend
GET	`/api/v1/knowledge-base/benchmarks`	ADMIN	Cross-project duration/productivity benchmarks, filterable by discipline/activity type
POST	`/webhooks/pmis/sync-ack`	SYSTEM	Primavera P6 / MS Project delivery acknowledgment callback

Reconciled against the working prototype: `/` , `/health` , `/schedule/upload` , `/schedule/activities` , `/progress/upload-excel` , `/progress/events` , and `/agent/sessions/{session_id}/events` were present in the implementation but missing from v2.1.0's API design — added here. `/api/v1/schedule/matches/{match_id}/resolve` was in the design but absent from the prototype's current endpoint list — it should not be dropped, since it's what closes the human-in-the-loop review workflow described in §6.4.

8.5 API Design Principles

- **Idempotency:** `event_id` is client-generatable (UUID) and enforced unique — safe retries on flaky site connectivity never create duplicate events.
- **Versioning:** functional endpoints are URI-versioned (`/api/v1/...`); breaking changes ship as `/api/v2` with a deprecation window, never in place. `/` and `/health` stay unversioned, per standard infra convention — orchestrators and load balancers expect a stable, un-prefixed probe path.
- **Pagination:** all list endpoints (`schedule/activities` , `progress/events` , `schedule/queue`) are cursor-paginated to keep dashboards responsive as data volume grows.
- **Batch vs. single ingestion:** `events/ingest` handles one field update; `progress/upload-excel` handles many — kept as separate endpoints rather than overloading one, since their validation, response shape, and failure-partial-batch handling differ.
- **Consistent error envelope:** RFC 7807 `application/problem+json` across all 4xx/5xx responses.

9. Database Schema (PostgreSQL + pgvector)

Extends the v2.0 schema to support discipline/contractor scoping, nullable matches for new-activity discovery, partial-progress rollup, the active-learning glossary, and cross-project institutional memory. Additions are marked **NEW**; nothing from v2.0 is removed.

```
-- 1. EXTENSIONS
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 2. ENUMS
CREATE TYPE routing_status AS ENUM
  ('AUTO_COMMITTED','PENDING_REVIEW','NEW_ACTIVITY_CANDIDATE','MANUAL_COMMITTED','REJECTED');
CREATE TYPE discipline_type AS ENUM

('CIVIL','PIPING','STATIC_EQUIPMENT','ROTATING_EQUIPMENT','ELECTRICAL','INSTRUMENTATION','HSE','OTHER');
CREATE TYPE user_role AS ENUM
  ('SUPERVISOR','PLANNER','ADMIN','CONTRACTOR_ADMIN','SYSTEM');

-- NEW: ingestion_sources is a lookup TABLE, not a hard ENUM, so a future
-- CCTV_VISION or other modality is a data insert -- never a schema migration.
CREATE TABLE ingestion_sources (
    source_code VARCHAR(30) PRIMARY KEY,
    description TEXT
);
INSERT INTO ingestion_sources (source_code, description) VALUES
  ('VOICE_APP', 'One-shot voice note, Whisper ASR'),
  ('TIME_AGENT', 'Conversational voice/chat agent session'),
  ('TEXT_DIARY', 'Free-text daily report / site diary'),
  ('SPREADSHEET', 'Discipline-wise Excel/CSV report'),
  ('PDF_OCR', 'Scanned document, OCR extracted'),
  ('PMIS_EXPORT', 'Primavera/MS Project export ingested for reconciliation');
  -- Future: ('CCTV_VISION', 'Computer-vision detected activity state') -- see Roadmap

-- 3. DOMAIN TABLES
CREATE TABLE organizations (                                    -- NEW: contractor/company
scoping
    org_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
```

```
    org_type VARCHAR(50) NOT NULL,                              -- OWNER | EPC | SUBCONTRACTOR
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE users (                                            -- NEW: explicit identity for
RBAC + reported_by
    user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    org_id UUID REFERENCES organizations(org_id),
    role user_role NOT NULL,
    discipline discipline_type,
    display_name VARCHAR(150) NOT NULL,
    preferred_language VARCHAR(20) DEFAULT 'en',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE projects (
    project_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_code VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    status VARCHAR(50) DEFAULT 'ACTIVE',
    auto_commit_threshold NUMERIC(4,3) DEFAULT 0.850,           -- NEW: per-project
configurable
    min_candidate_threshold NUMERIC(4,3) DEFAULT 0.600,         -- NEW: floor below which ->
NEW_ACTIVITY_CANDIDATE
    score_alpha NUMERIC(3,2) DEFAULT 0.70,                      -- NEW: S_final = alpha*S_dense
+ beta*S_symbolic
    score_beta NUMERIC(3,2) DEFAULT 0.30
);

CREATE TABLE wbs_nodes (
    wbs_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID REFERENCES projects(project_id) ON DELETE CASCADE,
    activity_code VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    discipline discipline_type NOT NULL,
    actual_start TIMESTAMPTZ,
    actual_end TIMESTAMPTZ,
    percent_complete NUMERIC(5,2) DEFAULT 0.00,
    predecessors JSONB DEFAULT '[]'::jsonb,
    description_vector vector(1536),
    is_planner_created BOOLEAN DEFAULT FALSE,                   -- NEW: promoted from a new-
activity candidate
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (project_id, activity_code)
);
CREATE INDEX idx_wbs_vector ON wbs_nodes USING hnsw (description_vector vector_cosine_ops);
CREATE INDEX idx_wbs_project ON wbs_nodes (project_id);
CREATE INDEX idx_wbs_discipline ON wbs_nodes (project_id, discipline);   -- NEW: discipline
dashboards

-- 4. INGESTION & ROUTING
CREATE TABLE field_events (
    event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),       -- client-generated,
idempotency key
    project_id UUID REFERENCES projects(project_id) ON DELETE CASCADE,
    reported_by UUID REFERENCES users(user_id) NOT NULL,
    org_id UUID REFERENCES organizations(org_id),                -- NEW: contractor scoping
    discipline discipline_type,                                  -- NEW: known even before a
match is found
    source_type VARCHAR(30) REFERENCES ingestion_sources(source_code) NOT NULL,
```

```
    raw_input_text TEXT NOT NULL,
    extracted_json JSONB NOT NULL,
    reported_timestamp TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_events_project_discipline ON field_events (project_id, discipline);

CREATE TABLE event_wbs_matches (
    match_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    event_id UUID REFERENCES field_events(event_id) ON DELETE CASCADE,
    wbs_id UUID REFERENCES wbs_nodes(wbs_id) ON DELETE CASCADE NULL,  -- CHANGED: nullable
    match_type VARCHAR(30) NOT NULL DEFAULT 'EXISTING_NODE',          -- NEW: EXISTING_NODE |
NEW_ACTIVITY_CANDIDATE
    progress_contribution_pct NUMERIC(5,2),                           -- NEW: granularity/
partial-progress rollup
    semantic_score NUMERIC(4,3) NOT NULL,
    symbolic_score NUMERIC(4,3) NOT NULL,
    confidence_score NUMERIC(4,3) NOT NULL,
    status routing_status NOT NULL DEFAULT 'PENDING_REVIEW',
    reviewed_by UUID REFERENCES users(user_id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_planner_queue ON event_wbs_matches(project_id, status)
  WHERE status IN ('PENDING_REVIEW','NEW_ACTIVITY_CANDIDATE');

-- NEW: active-learning glossary, backing Feature 6.5
CREATE TABLE glossary_mappings (
    glossary_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID REFERENCES projects(project_id) ON DELETE CASCADE,
    discipline discipline_type NOT NULL,
    field_term TEXT NOT NULL,
    wbs_activity_code VARCHAR(100) NOT NULL,
    source_match_id UUID REFERENCES event_wbs_matches(match_id),
    confidence NUMERIC(4,3),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- NEW: institutional knowledge base (Feature 6.6) -- cross-project by design
CREATE TABLE delay_reasons (
    delay_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID REFERENCES projects(project_id),
    wbs_id UUID REFERENCES wbs_nodes(wbs_id),
    discipline discipline_type NOT NULL,
    reason_category VARCHAR(80) NOT NULL,     -- e.g. MATERIAL_DELAY, WEATHER, REWORK, PERMIT
    reason_text TEXT,
    days_impact NUMERIC(6,2),
    logged_by UUID REFERENCES users(user_id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE productivity_benchmarks (
    benchmark_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    source_project_id UUID REFERENCES projects(project_id),
    discipline discipline_type NOT NULL,
    activity_type VARCHAR(150) NOT NULL,       -- normalized activity class, not raw
description
    planned_duration_days NUMERIC(6,2),
    actual_duration_days NUMERIC(6,2),
    crew_size INTEGER,
    computed_at TIMESTAMPTZ DEFAULT NOW()
```

```
);
CREATE INDEX idx_benchmarks_lookup ON productivity_benchmarks (discipline, activity_type);  --
cross-project query path

-- 5. AUDIT LOGGING (Immutable)
CREATE TABLE audit_logs (
    audit_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID NOT NULL,
    action VARCHAR(50) NOT NULL,
    old_state JSONB,
    new_state JSONB,
    performed_by VARCHAR(100) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

> **Why `ingestion_sources` is a table, not an enum:** Postgres enums require a schema migration (`ALTER TYPE ... ADD VALUE` , with its own transactional quirks) to add a value. A lookup table lets the future CCTV/vision adapter (§20) register itself with an `INSERT` — no migration, no downtime, no coordination with the core schema owner.

10. Enterprise Security Architecture (Zero-Loophole Standard)

Defense-in-depth, adhering strictly to OWASP Top 10 mitigation strategies.

10.1 Authentication & Authorization

- **Stateless Auth:** short-lived JWTs (15-minute expiry) with opaque refresh tokens stored in Redis, rotated on every use (rotation invalidates the prior token, limiting replay).
- **Token Transport:** delivered to the frontend exclusively via `HttpOnly` , `Secure` , `SameSite=Strict` cookies — eliminates XSS token theft. LocalStorage is strictly forbidden for token storage.
- **CSRF:** double-submit CSRF token required on all state-changing (POST/PUT/PATCH/DELETE) requests, since cookie-based auth alone does not eliminate CSRF.
- **RBAC:** FastAPI dependency injection enforces roles (`@requires_role(["PLANNER","ADMIN"])`) plus `organization_id` / `discipline` scoping before route execution.
- Option OAuth 2.0 + OpenID Connect (OIDC) as the identity layer for enterprise SSO integration, issuing the internal JWT after federation.

10.2 Network & Infrastructure Security

- TLS 1.3 enforced for all client-to-server and server-to-server (including gRPC/webhook) communication.
- IP- and user-based rate limiting via Redis at the FastAPI middleware layer to prevent DDoS and brute-force attacks.
- Strictly whitelisted CORS origins — no wildcard (`*`) headers.

10.3 Data Security & Injection Prevention

- **SQL Injection:** zero string-concatenation SQL — all interactions via SQLAlchemy ORM/Core parameterized queries.

- **Prompt Injection:** input sanitization and strict system-prompt boundaries on the LLM extractor and Time-Agent, so a supervisor's free text cannot override the extraction schema or agent instructions.
- **Data at Rest:** AES-256 encryption on PostgreSQL disk volumes, object stores, and vector caches (cloud KMS-managed).
- **Idempotency:** every ingestion request carries a client-generated `event_id` (equivalently an `X-Idempotency-Key` header) so retried requests on flaky site connectivity can never create duplicate schedule updates.

10.4 Access Control Matrix

Role	Permissions
Site Supervisor	Create/write field progress events (ingestion, Time-Agent); read own submitted history
Planner	Read progress events and matches within assigned discipline(s)/project; approve, correct, or reject matches; promote new-activity candidates to WBS nodes
Project Controls / Admin	Cross-discipline read access; analytics and forecasting; knowledge-base queries
Contractor Admin	Manage users within own organization; scoped read access to own organization's data
System Admin	Full configuration, API key lifecycle, audit log access, threshold tuning

11. Non-Functional Requirements

11.1 Performance & Scalability

Requirement	Target
Text ingestion response time	< 500ms
Voice-to-JSON parsing (audio up to 60s)	< 3 seconds
Ingest endpoint acknowledgment (202 Accepted)	< 50ms p99
Peak ingestion throughput	10,000 progress entries / minute during shift-change windows
Autoscaling trigger	Horizontal Pod Autoscaling at 70% CPU/memory utilization (Kubernetes)
Vector search latency at scale	Millisecond-range nearest-neighbor search via HNSW, even at millions of historical activities

11.2 Reliability & Fault Tolerance

- **Availability target:** 99.95% uptime for the ingestion and matching pipeline.
- **Dead-Letter Queue (DLQ):** events failing extraction or matching after 3 retries move to an isolation queue for inspection, without blocking the main pipeline.
- **Idempotency:** enforced at the API layer via `event_id` uniqueness — see §10.3.
- **Graceful degradation:** if the LLM extraction service is unavailable, raw input is still persisted and queued for retry rather than rejected.

12. High-Scale Optimization Techniques

1. **Connection Pooling:** PgBouncer in front of PostgreSQL prevents connection starvation during traffic spikes, routing thousands of concurrent FastAPI requests through a bounded pool.
2. **Vector Search Optimization:** pgvector's HNSW index changes nearest-neighbor search from O(N) to O(log N).
3. NEW **Embedding Quantization:** compress 1536-dimensional FP32 embeddings to INT8 for the hot cache path, cutting memory overhead by roughly 75% with minimal recall loss — full-precision vectors remain the source of truth in pgvector.
4. **Asynchronous I/O:** FastAPI's `async def` + `asyncpg` for all direct database reads (e.g. the Planner Dashboard) so the server thread never blocks on network I/O.
5. **Read-Heavy Caching:** the baseline WBS schedule (rarely changes mid-shift) is cached in Redis with a 24-hour TTL; Celery workers hit the cache first for symbolic constraint validation.

13. Observability & Monitoring

Layer	Approach
Structured logging	JSON logs correlated by `event_id` across API → queue → worker → DB write, shipped to a central log store
Metrics	Prometheus counters/histograms: ingest latency, queue depth, auto-commit rate, DLQ size, matching confidence distribution
Tracing	OpenTelemetry spans across the async boundary (API → Redis → Celery → Postgres) so a single field event can be traced end to end
Alerting	Queue depth breach, DLQ growth rate, auto-commit rate drop (signals matching-quality regression), PMIS sync failures
Dashboards	Grafana: system health (ops) + discipline-wise progress and delay dashboards (PMO)

14. Verification & Test Plan

14.1 Test Pipeline

Unit tests (extraction schema, constraint solver rules) → Embedding/matching accuracy benchmarks → End-to-end ingestion-to-commit tests → Performance/load benchmark verification → Planner Review Queue UI validation.

14.2 Benchmark Test Cases

Test Case	Input	Context	Expected Result
TC-001 Vocabulary mismatch	Voice text: "Erected spool line 12 zone 4."	Target node: `WBS-L6-PIP-0914` — "Erect Steel Piping Line 12 – Area 4"	$S_{final} \geq 0.88$ → auto-routed to schedule update

TC-002 Precedence violation	Field text: "Completed final painting for Line 14."	WBS state shows `WBS-L6-PIP-0913` (Hydrostatic Test, Line 14) still incomplete	Symbolic pass fails the match; routes to Planner Review Queue with warning "Precedence Constraint Violation"
TC-003 Granularity mismatch **NEW**	Three separate field events across a shift: "welded joint 1," "welded joint 2," "welded joint 3" on the same line	Single WBS node covers the full weld-out	All three roll up against one node with summed `progress_contribution_pct`, not three competing matches
TC-004 No matching node **NEW**	"Installed temporary access scaffold near Zone 7"	No WBS node describes temporary scaffolding	$S_{final} < 0.60$ → routed to `NEW_ACTIVITY_CANDIDATE` queue, not dropped

15. Deployment & CI/CD

- **Environments:** dev → staging → production, with staging mirroring production's PgBouncer/Redis/Celery topology at reduced scale.
- **Pipeline:** lint + unit tests + embedding-accuracy regression suite on every PR; container build and vulnerability scan; blue-green deploy to Kubernetes.
- **Database migrations:** versioned via Alembic, applied automatically in CI before service rollout, backward-compatible for one release to support zero-downtime deploys.
- **Secrets management:** cloud KMS / Kubernetes Secrets — never committed, never baked into images.

16. File Structure & Monorepo Layout

The repository is structured as a monorepo containing the React frontend, FastAPI backend, and infrastructure configuration.

```
infrastructure-sync-monorepo/
├── frontend/                    # React.js (Vite + TypeScript)
│   ├── src/
│   │   ├── components/          # Reusable UI (Buttons, Modals)
│   │   ├── features/            # Domain-specific modules
│   │   │   ├── planner-queue/   # Exception Dashboard + New-Activity Queue
│   │   │   ├── time-agent/      # NEW: conversational capture UI
│   │   │   ├── dashboard/       # Analytics & Knowledge Graph UI
│   │   │   └── ingestion/       # File upload & voice recording UI
│   │   ├── hooks/               # Custom React hooks (e.g., useAuth)
│   │   ├── services/            # Axios API clients
│   │   └── store/               # Global state (Zustand or Redux)
│   ├── package.json
│   └── vite.config.ts
│
├── backend/                     # FastAPI (Python 3.11+)
│   ├── app/
│   │   ├── api/v1/
│   │   │   ├── auth.py          # JWT generation & validation
│   │   │   ├── events.py        # Ingestion endpoints (fast 202 Accepted)
│   │   │   ├── agent.py         # NEW: Time-Agent conversational session
│   │   │   └── schedule.py      # Planner queue read/write
│   │   ├── core/                # Security, Config, Redis setup
```

```
│   │   │   ├── config.py
│   │   │   └── security.py         # Password hashing, JWT auth, RBAC
│   │   ├── db/
│   │   │   ├── models.py           # Maps to SQL schema (§9)
│   │   │   └── session.py          # asyncpg connection pooling
│   │   ├── schemas/                # Pydantic models for validation
│   │   ├── services/
│   │   │   ├── llm_extractor.py    # LLM function calling
│   │   │   ├── constraint_solver.py# Symbolic rule checking
│   │   │   └── ingestion_adapters/ # NEW: pluggable per-modality adapters
│   │   └── workers/                # Celery background tasks
│   │       ├── celery_app.py
│   │       └── matching_tasks.py   # Async vector search & confidence routing
│   ├── requirements.txt
│   └── main.py
│
└── infrastructure/
    ├── docker-compose.yml          # Local dev (Postgres, Redis, Celery, API, Web)
    ├── k8s/                        # Kubernetes deployment manifests
    └── terraform/                  # Cloud infrastructure provisioning
```

17. Risks & Mitigations

Risk	Severity	Mitigation
LLM extraction misreads discipline-specific jargon at launch, before glossary matures	High	Conservative default thresholds at go-live; Time-Agent clarifying questions; fast planner-correction feedback loop
Site connectivity drops mid-submission	Medium	Client-side retry with idempotent `event_id`; queued local draft in the mobile app
Planner queue backs up faster than it clears	High	Per-discipline queue routing, SLA alerting, threshold auto-tuning as glossary confidence improves
PMIS (Primavera/MS Project) write-back conflicts with a concurrent manual schedule edit	High	Optimistic concurrency check against PMIS's own revision/version stamp before write; conflict routed to planner, never silently overwritten
Multi-contractor data visibility disputes	Medium	Strict `organization_id` + discipline scoping in RBAC, audited access logs
Vector index staleness as WBS is re-baselined mid-project	Low	Embedding regeneration triggered on WBS description change via DB trigger → Celery task

18. Problem-to-Solution Traceability Matrix

Problem Statement Requirement	Addressed By
Multi-discipline parallel execution & reporting	§6.7 Discipline/Contractor RBAC; `discipline_type` enum across schema (§9)

Low-friction conversational/voice logging	§6.2 Conversational Time-Agent Interface
Terminology differences across disciplines	§6.3 Semantic Pass (pgvector cosine similarity)
Field granularity finer than WBS	§6.3 Granularity Reconciliation, `progress_contribution_pct`
Flag unmatched/new activities instead of dropping	§6.4 New-Activity Queue (`NEW_ACTIVITY_CANDIDATE`), TC-004
Near-real-time PMIS update with confidence + audit trail	§7.1 Confidence-routing flow; §9 `audit_logs` ; §10.4 access control
Discipline-tagged dataset for analytics/ forecasting	§8.1 `/analytics/discipline-summary` ; §13 dashboards
Institutional memory, queryable across projects	§6.6 Institutional Knowledge Graph; §9 `delay_reasons` , `productivity_benchmarks`
Multi-contractor data ownership	§9 `organizations` table; §10.4 access control matrix
Future CCTV data as an additional input	§20 Roadmap — pluggable ingestion adapter pattern, §9 `ingestion_sources` lookup table

19. Roadmap & Future Work

19.1 Phase 2 — Computer-Vision (CCTV) Ingestion Adapter

The architecture is deliberately designed so this does not require re-architecture when it's built:

- CCTV frame samples are processed by a vision model (object/state detection — e.g., "concrete poured," "equipment installed," "scaffold erected") running as its own Celery worker pool.
- Detections are converted into the **same standardized extraction JSON** used by every other modality (§6.1), tagged with `source_type = 'CCTV_VISION'` — a row insert into `ingestion_sources` , not a schema migration.
- From that point on, the event flows through the identical neuro-symbolic matching engine, confidence router, and audit pipeline as a voice note or spreadsheet row.
- Camera-derived events would additionally carry a `location_zone` and timestamp range with high confidence, which can raise matching precision for physically observable milestones (concrete pours, structural erection) beyond what text-only reporting achieves.

19.2 Other Candidate Enhancements

- Offline-first mobile capture with local queueing for zero-connectivity sites.
- Natural-language / conversational query interface over the institutional knowledge base (e.g., "P90 actual duration for 24-inch piping installation in wet weather, past projects") — a natural extension of §6.6 once benchmark data density is sufficient.
- Automated delay-reason classification from free text, reducing manual tagging in `delay_reasons` .

20. Appendix — Glossary

Term	Definition
WBS	Work Breakdown Structure — hierarchical decomposition of project scope; L5/L6 are the most granular, executable activity levels.
PMIS	Project Management Information System — e.g., Primavera P6, MS Project.
HNSW	Hierarchical Navigable Small World — an approximate nearest-neighbor index structure used by pgvector for fast vector search.
Confidence Score (S_{final})	Weighted combination of semantic (vector) and symbolic (rule-based) match scores, used to route an event to auto-commit, review, or new-activity discovery.
Time-Agent	The conversational LLM interface supervisors use to log progress via voice or chat.
DLQ	Dead-Letter Queue — isolation queue for events that repeatedly fail extraction or matching, so they never block the main pipeline nor get silently lost.
Active Learning Glossary	Structured store of human-corrected field-term → WBS-activity-code mappings, used to improve future extraction accuracy.